

# ソフトウェア演習 1 レポート 5

群馬大学情報学部 3 年

伊藤 駿

## 1. ソースコード

ソースコードは以下に格納されている。

<https://github.com/Hayao0819/zash/tree/0564f6f3ec190de74605541aa3f7df1063305cad/go>

## 2. 本レポートについて

本レポートは Typst を用いて作成されている。

以下のリンクでレポートのソースコード及びビルド結果について確認できる。

<https://typst.app/project/rw1VwQUoKwL98PjRSEkVZK>

## 3. 用語説明

### 3.1. シェル

シェルは、ユーザーと OS（オペレーティングシステム）の間を仲介するプログラムのことである。ユーザーがキーボードから入力したコマンドを解釈し、OS の機能やアプリケーションを起動・制御する役割を担っている。

シェルは大きく分けて 2 種類ある。ひとつはコマンドをテキストで入力する CUI（Character User Interface）シェルで、Bash や Zsh などがこれにあたる。もうひとつはマウス操作でアイコンをクリックする GUI（Graphical User Interface）シェルで、Windows のエクスプローラーや macOS の Finder がこれにあたる。プログラミングやシステム管理の分野では、主に CUI シェルを指すことが多い。

### 3.2. システムコール

システムコールは、アプリケーションプログラムが OS のカーネル（OS の中核部分）が持つ機能を利用するための窓口だ。アプリケーションは、メモリ管理、ファイルアクセス、デバイス制御など、OS の特権的な機能に直接アクセスすることはできない。代わりに、特定の関数や命令を呼び出すことで、カーネルに処理を依頼する。

この「依頼」がシステムコールであり、アプリケーションはカーネルモードの特権的な操作を直接実行する代わりに、安全にその恩恵を受けられる。たとえば、ファイルを開く `open()` や、データを読み込む `read()` などが代表的なシステムコールとなる。

### 3.3. Intel SGX

Intel SGX は、CPU がエンクレーブと呼ばれる特別なメモリ領域を作成することを可能にする。エンクレーブ内のコードとデータは、OS やハイパーバイザを含むシステム上の他のすべてのソフトウェアから隔離され、保護される。たとえ、管理者権限を持つソフトウェアがエンクレーブのメモリにアクセスしようとしても、CPU はこれを拒否する。これにより、機密性の高い処理（例：暗号鍵の生成、個人情報の処理）を安全な環境で行うことができる。

## 4. 作成したもの

Go 言語を用いて CLI のインタラクティブシェルを実装しようとした。しかし、実装時間や技術的な都合により最後まで実装できなかった。

本レポートでは実装しようとした機能と、実際に実装したもの・実装できなかったものについてまとめる。

### 4.1. 当初実装したかったもの

当初、計画していた要件は以下の通りである。

- 常に標準入力を待つ
- 入力された文字列から AST を構築
- 組み込みコマンドを実行したり、起動するバイナリ、引数、出力先、環境変数を決定しシステムコールを発行する

また、シェルとしての機能を完全なものとするため、以下の構文や機能も実装する。

- ファイルをオープンし内容を読み取る
- 読み取った内容から AST を構築
- 組み込みコマンドを実行したり、起動するバイナリ、引数、出力先、環境変数を決定しシステムコールを発行する
- if 制御文による条件分岐
- while 制御文によるループ処理
- シェル変数のパースと展開

また、千田研究室で勉強中の知識を活かしたいと考え、Intel SGX を用いて以下の機能を実装する。

- 変数として機密情報を保持
- 状態は暗号化して Sealing
- 起動時に Unsealing し、必要なプロセスにのみ情報を渡す

実装されたシェルを用いて本授業で使用されている投票システムに対し DoS 攻撃を行う。昨年度に同様の授業を受講した先輩から攻撃しても良いという旨を聞いていたので、これ为目标に開発を行った。

### 4.2. 最終的に実装したもの

#### 4.2.1. インタラクティブシェル

- 標準入力からの受取
- 入力文字列のパース(文字列、"による文字列区切り、空白による引数の分解)
- バイナリの実行
- リダイレクト

環境変数の操作については実装できなかった。

#### 4.2.2. シェルスクリプトの実行

- ファイルを第一引数で渡して実行
- 基本的なコマンド呼び出し
- ファイルへのリダイレクト
- while によるループ
- コメントのパース

シェル変数や if 文による制御は実装できなかった。また、パイプによるプロセス間通信も実装できなかった。

#### 4.2.3. Intel SGX による秘密保護

当初 Edgeless Systems 社による EGO(<https://www.edgeless.systems/products/ego>)を用いて実装しようとしていたが、レポート締め切りまでに Intel SGX の動作環境の構築を行うことができなかった。

Intel SGX の Enclave の起動に関して奮闘した記録は以下のスライドに掲載されている。

<https://www.docswell.com/s/hayao/ZEYWMG-sgx-driver>

## 5. 動作している様子

```
hayao@Hayao-XPS9350 ~/G/z/go (master)> go run _ -h
Zash is a shell written in Go.

Usage:
  zash [script_file] [flags]
  zash [command]

Available Commands:
  completion  Generate the autocompletion script for the specified shell
  help        Help about any command
  run         Run a command

Flags:
  -c, --command string  command to execute
  -d, --debug            debug mode
  -h, --help            help for zash

Use "zash [command] --help" for more information about a command.
hayao@Hayao-XPS9350 ~/G/z/go (master)> 
```

```
hayao@Hayao-XPS9350 ~/G/z/go (master)> go run _ run "echo hello shell"
hello shell
hayao@Hayao-XPS9350 ~/G/z/go (master)> 
```

```
Wanya? | hayao | ~/Git/zash/go > seq 1 3
1
2
3
Wanya? | hayao | ~/Git/zash/go > cat /etc/os-release
NAME="Arch Linux"
PRETTY_NAME="Arch Linux"
ID=arch
BUILD_ID=rolling
ANSI_COLOR="38;2;23;147;209"
HOME_URL="https://archlinux.org/"
DOCUMENTATION_URL="https://wiki.archlinux.org/"
SUPPORT_URL="https://bbs.archlinux.org/"
BUG_REPORT_URL="https://gitlab.archlinux.org/groups/archlinux/-/issues"
PRIVACY_POLICY_URL="https://terms.archlinux.org/docs/privacy-policy/"
LOGO=archlinux-logo
Wanya? | hayao | ~/Git/zash/go > █
```

[illegible]

## 6. ベンチマーク

今回作成した Zash と最も利用されている Bash でスクリプトの実行時間を比較する。

## 6.1. 方法

ベンチマークには以下の `echo` を実行するシンプルなスクリプトを複数回実行し、実行時間を比較する。

```
1 echo Hello1
2 echo Hello2
3 echo Hello3
4 echo Hello4
5 echo Hello5
6 echo Hello6
7 echo Hello7
8 echo Hello8
9 echo Hello9
10 echo Hello10
```

```
11 echo Hello11
12 echo Hello12
13 echo Hello13
14 echo Hello14
15 echo Hello15
16 echo Hello16
17 echo Hello17
18 echo Hello18
19 echo Hello19
20 echo Hello20
21 echo Hello21
22 echo Hello22
23 echo Hello23
24 echo Hello24
25 echo Hello25
26 echo Hello26
27 echo Hello27
28 echo Hello28
29 echo Hello29
30 echo Hello30
31 echo Hello31
32 echo Hello32
33 echo Hello33
34 echo Hello34
35 echo Hello35
36 echo Hello36
37 echo Hello37
38 echo Hello38
39 echo Hello39
40 echo Hello40
41 echo Hello41
42 echo Hello42
43 echo Hello43
44 echo Hello44
45 echo Hello45
46 echo Hello46
47 echo Hello47
48 echo Hello48
49 echo Hello49
50 echo Hello50
51 echo Hello51
52 echo Hello52
53 echo Hello53
54 echo Hello54
55 echo Hello55
56 echo Hello56
57 echo Hello57
58 echo Hello58
59 echo Hello59
60 echo Hello60
61 echo Hello61
62 echo Hello62
63 echo Hello63
64 echo Hello64
65 echo Hello65
66 echo Hello66
67 echo Hello67
68 echo Hello68
69 echo Hello69
70 echo Hello70
71 echo Hello71
72 echo Hello72
```

```
73 echo Hello73
74 echo Hello74
75 echo Hello75
76 echo Hello76
77 echo Hello77
78 echo Hello78
79 echo Hello79
80 echo Hello80
81 echo Hello81
82 echo Hello82
83 echo Hello83
84 echo Hello84
85 echo Hello85
86 echo Hello86
87 echo Hello87
88 echo Hello88
89 echo Hello89
90 echo Hello90
91 echo Hello91
92 echo Hello92
93 echo Hello93
94 echo Hello94
95 echo Hello95
96 echo Hello96
97 echo Hello97
98 echo Hello98
99 echo Hello99
100 echo Hello100
```

このスクリプトを Rust 製のベンチマークツール Hyperfine を用いて複数回実行し、ベンチマークを行う。

Hyperfine は以下で公開されている。

<https://github.com/sharkdp/hyperfine>

これらの処理を自動化したスクリプトが以下である。

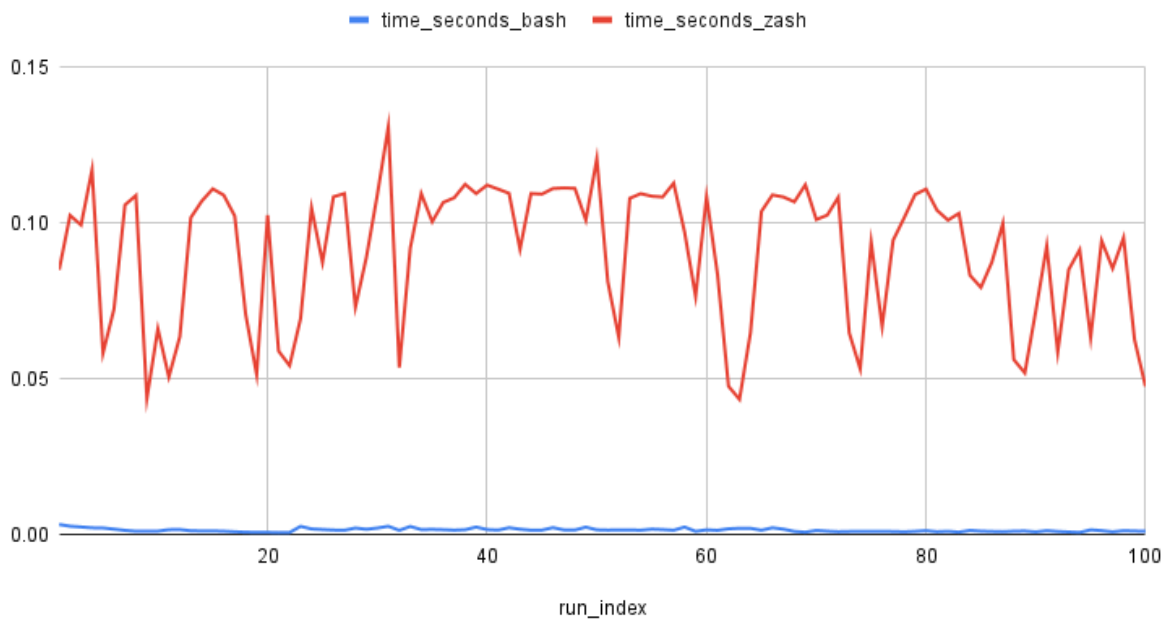
## 6.2. 結果

```
Hello95
Hello96
Hello97
Hello98
Hello99
Hello100
Time (mean ± σ):    71.1 ms ± 16.8 ms    [User: 34.9 ms, System: 36.6 ms]
Range (min ... max): 32.0 ms ... 117.5 ms    100 runs

Summary
/usr/bin/bash /home/hayao/Git/hayao-tools/univ/software-exercise/final-report/benchmark/100-hello.sh ran
53.19 ± 25.53 times faster than /usr/local/bin/zash /home/hayao/Git/hayao-tools/univ/software-exercise/final-report/benchmark/1
Done. CSV saved to: /home/hayao/Git/hayao-tools/univ/software-exercise/final-report/benchmark/benchmark.csv
JSON saved to: /home/hayao/Git/hayao-tools/univ/software-exercise/final-report/benchmark/benchmark.json
Per-run CSV for bash saved to: /home/hayao/Git/hayao-tools/univ/software-exercise/final-report/benchmark/result-detail-bash.csv
Per-run CSV for zash saved to: /home/hayao/Git/hayao-tools/univ/software-exercise/final-report/benchmark/result-detail-zash.csv
Combined per-run CSV saved to: /home/hayao/Git/hayao-tools/univ/software-exercise/final-report/benchmark/benchmark-runs.csv
hayao@Hayao-XPS9350 ~/G/h/u/s/f/benchmark (master)>
```

Bash のほうが圧倒的に高速であり、私の作成したものは数百倍も遅かった。

## time\_seconds\_bash と time\_seconds\_zash



赤軸が私の作成したシェルの実行時間であり、下の青線が Bash の実行時間である。

## 7. 考察

### 7.1. 実装が間に合わなかった理由

かなり初期の頃から実装を進めていたが、パーサーとレクサーの実装にかなりの時間がかかった。

結局当初の実装は全て破棄し、BNF をベースにパーサーを作り直した。

また、AST から実際にコマンドライン引数を組み立てるランタイム部分にかなり苦戦し、その実装は今も非常に汚くバグも多い状態になっている。

### 7.2. 実行時間の差について

Go 言語は C 言語に比較した際に非効率であり、バイナリサイズにも非常に差があることが要因として挙げられる。

詳細に実行時間を分析すると、スクリプトの読み取りと解析と実際の実行の双方で時間がかかっており根本的な構造がボトルネックとなっている。

## 8. 終わり

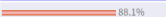
投票システムは非常に単純で CGI に対して POST リクエストを以下のように送れば投票数を増やせた。

```
1 while true; do
2     curl -X POST http://gadget.inf.gunma-u.ac.jp/cgi-bin/jikken1_2025/vote/vote.
3     cgi -d mode=vote -d number=9 -d no=1 2> /dev/null
4 done
```

これは私の稚拙なシェルでも実行可能な単純なものである。



実行時間のボトルネックによりあまり多くのリクエストは送信できなかったが、それでも最終的に 5555 票を投じることができた。

| ランク | 投票                       | 項目                                 | 投票数  | グラフ                                                                                       |
|-----|--------------------------|------------------------------------|------|-------------------------------------------------------------------------------------------|
| 1位  | <input type="checkbox"/> | J2300023 / 伊藤 駿 / シェル              | 5555 |  88.1% |
| 2位  | <input type="checkbox"/> | J2300501 / 萩原 康之 / KUE-CHIP2用アセンブラ | 60   | 11.0%                                                                                     |
| 3位  | <input type="checkbox"/> | J2300092 / 菅原 心暉 / midiファイル再生      | 45   | 10.7%                                                                                     |
| 4位  | <input type="checkbox"/> | J2300086 / 佐藤 悠斗 / スイカゲーム          | 43   | 10.7%                                                                                     |